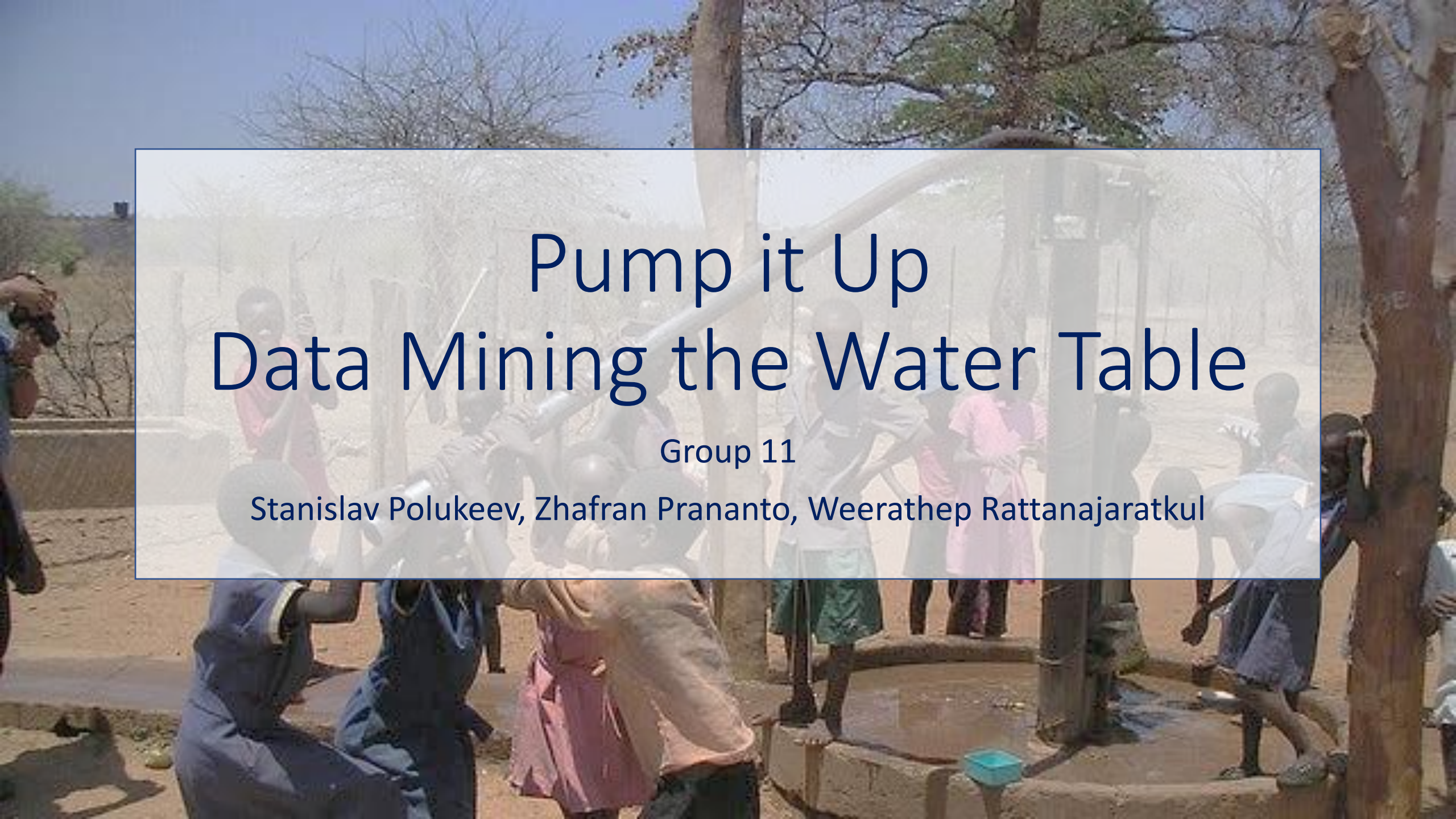


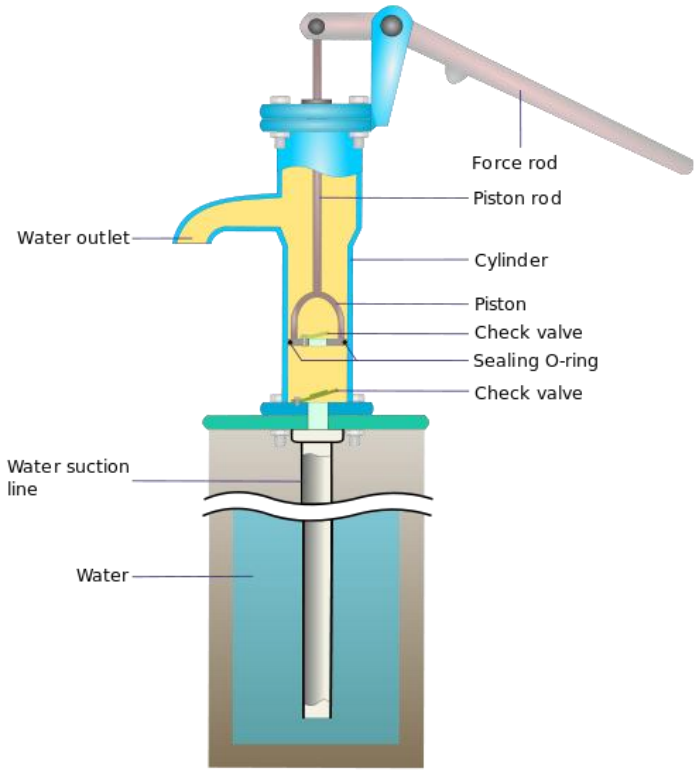
Pump it Up
Data Mining the Water Table

Group 11

Stanislav Polukeev, Zhafran Prananto, Weerathep Rattanajaratkul

Problem Statement

- Based on data published by the Tanzanian Ministry of Water and Taarifa, we must predict which water pumps are
  - Functional
  - Functional needs repair
  - Non-functional



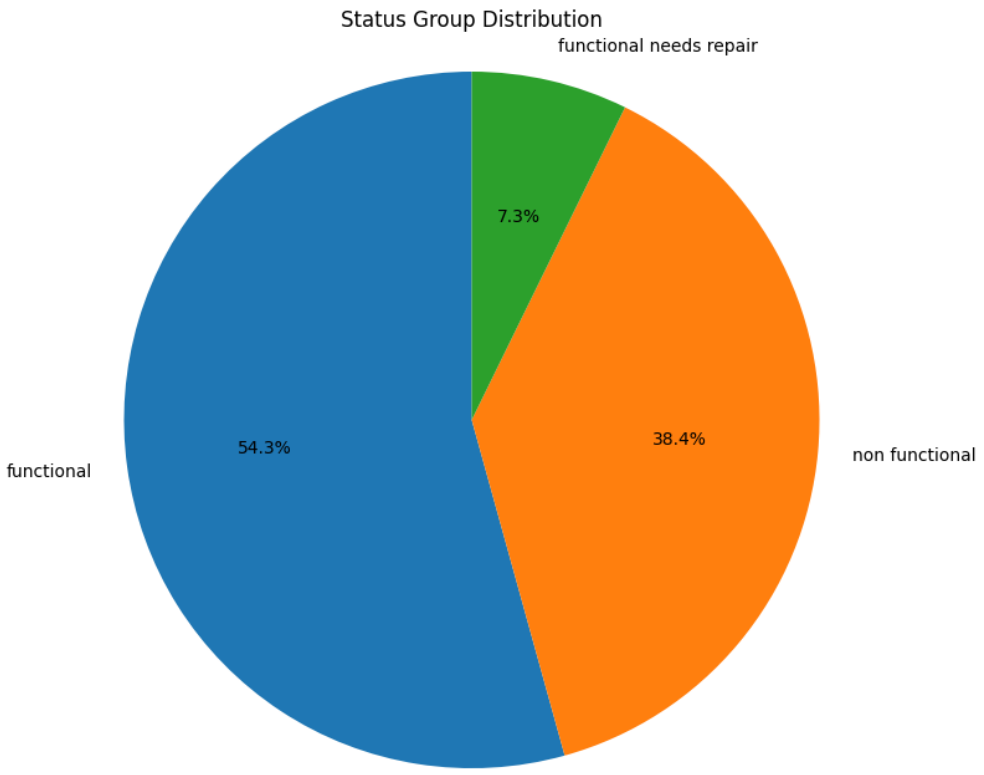
How can a ML model help?

- Aid maintenance operations
- Ensure safe and clean water is available to the rural communities of Tanzania

Dataset

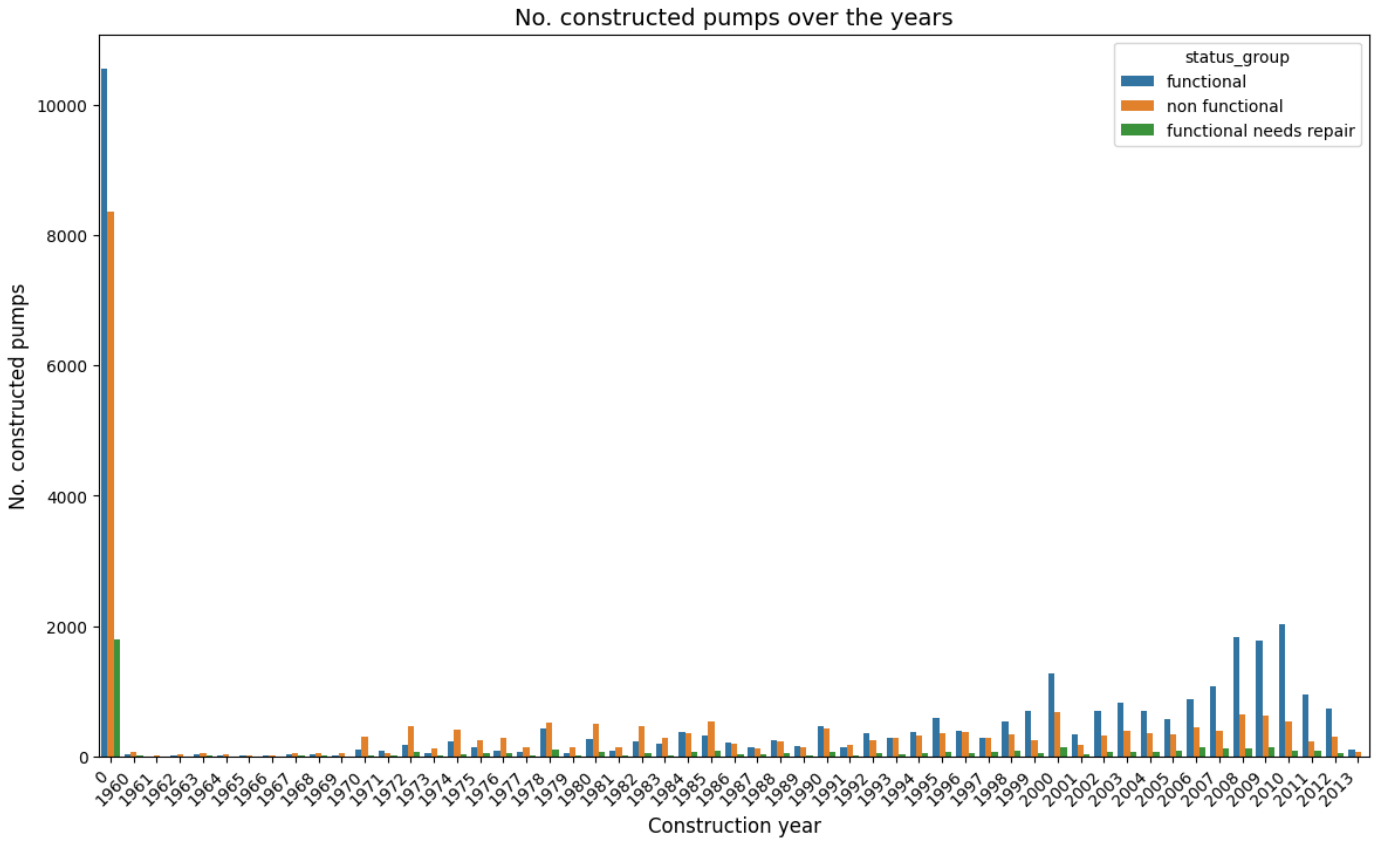
- 39 provided features and 59,400 records.
- A Class-imbalanced dataset

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 59400 entries, 0 to 59399
Data columns (total 41 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   id                     59400 non-null  int64
 1   amount_tsh             59400 non-null  float64
 2   date_recorded          59400 non-null  object
 3   funder                 55763 non-null  object
 4   gps_height             59400 non-null  int64
 5   installer              55745 non-null  object
 6   longitude              59400 non-null  float64
 7   latitude               59400 non-null  float64
 8   wpt_name               59398 non-null  object
 9   num_private            59400 non-null  int64
 10  basin                  59400 non-null  object
 11  subvillage             59029 non-null  object
 12  region                 59400 non-null  object
 13  region_code            59400 non-null  int64
 14  district_code          59400 non-null  int64
 15  lga                    59400 non-null  object
 16  ward                   59400 non-null  object
 17  population             59400 non-null  int64
 18  public_meeting         56066 non-null  object
 19  recorded_by            59400 non-null  object
...
 39  waterpoint_type_group  59400 non-null  object
 40  status_group           59400 non-null  object
dtypes: float64(3), int64(7), object(31)
memory usage: 18.6+ MB
```



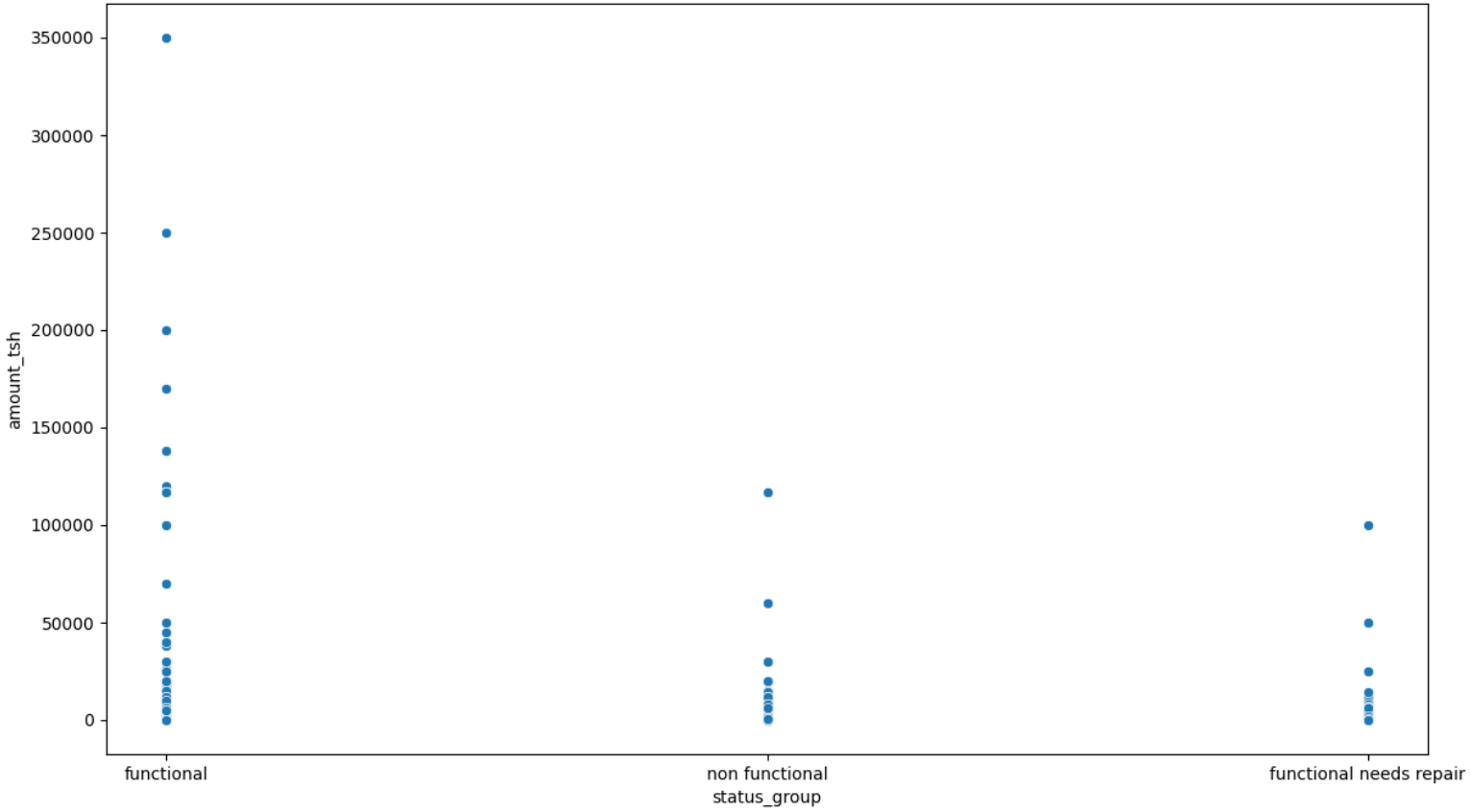
Exploratory Data Analysis

- Most of non-functional pumps were constructed in 1960.



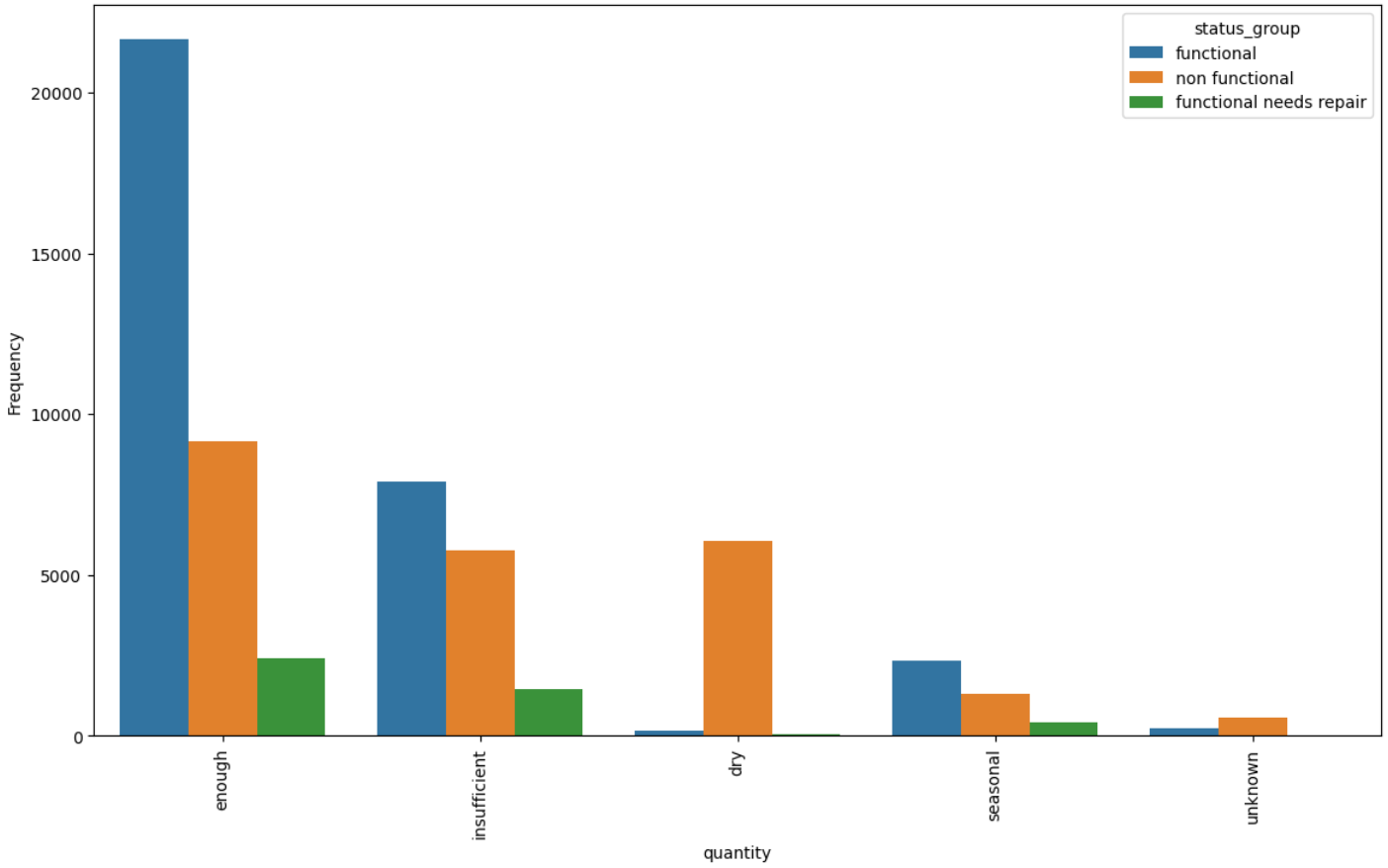
Exploratory Data Analysis

- If amount water > 15000, All pumps were functional.



Exploratory Data Analysis

- If water quantity is “Dry”, most of pumps were non-functional.



Prepare a dataset

- handle missing variables

```python
na_summary = (
    pd.DataFrame({
        'n_missing': df[str_features].isna().sum(),
        'nunique': df[str_features].nunique()
    })
    .query('n_missing > 0')
    .sort_values('n_missing', ascending=False)
)
na_summary
```

✓ 0.2s

	n_missing	nunique
scheme_name	28810	2695
scheme_management	3878	11
installer	3655	2145
funder	3637	1896
public_meeting	3334	2
permit	3056	2
subvillage	371	19287
wpt_name	2	37399

- Remove features : subvillage, wpt_name

  Because there are a lot of unique values compared to No. records

- Fill Nan with “Unknown” :

  scheme_name, scheme_management, installer, funder

- Fill Nan with Mode:

  public_meeting, permit

Prepare a dataset

- Keep one feature from similar features

The following groups of similar features

- (extraction_type, extraction_type_group, extraction_type_class) -> extraction_type
- (payment, payment_type) -> payment_type
- (water_quality, quality_group) -> water_quality
- (source, source_type , source_class) -> source
- (region, region_code, lga, ward) -> region_code
- (waterpoint_type, waterpoint_type_group) -> waterpoint_type
- (scheme_name, scheme_management) -> scheme_management

Prepare a dataset

- Create a “amount_tvsh_more_than_15000” feature that reflects EDA information.

```
def amount_tsh_more_than_15000(row):
    if row['amount_tsh'] > 15000:
        return 1
    else:
        return 0
df_filtered['amount_tsh_more_than_15000'] = df_filtered.apply(lambda row: amount_tsh_more_than_15000(row), axis=1)
```

- Create a “construction_year_cat” feature that groups into 10-year intervals.

```
def construction_wrangler(row):
    if row['construction_year'] >= 1960 and row['construction_year'] < 1970:
        return '60s'
    elif row['construction_year'] >= 1970 and row['construction_year'] < 1980:
        return '70s'
    elif row['construction_year'] >= 1980 and row['construction_year'] < 1990:
        return '80s'
    elif row['construction_year'] >= 1990 and row['construction_year'] < 2000:
        return '90s'
    elif row['construction_year'] >= 2000 and row['construction_year'] < 2010:
        return '00s'
    elif row['construction_year'] >= 2010 and row['construction_year'] < 2020:
        return '10s'
    else:
        return 'unknown'
df_filtered['construction_year_cat'] = df_filtered.apply(lambda row: construction_wrangler(row), axis=1)
```

Prepare a dataset

- Create a “date_recorded_diff” feature

  The idea : More recently recorded pumps might be more likely to be functional than non-functional.

```
df_filtered.date_recorded = pd.to_datetime(df_filtered.date_recorded)

df_filtered['date_recorded_diff'] = (pd.to_datetime('today') - pd.to_datetime(df_filtered.date_recorded)).dt.days
```

```
df_filtered['date_recorded_diff']
✓ 0.0s

0        5335
1        4612
2        4621
3        4649
4        5214
         ...
59395    4554
59396    5281
59397    5307
59398    5341
59399    5326
Name: date_recorded_diff, Length: 59400, dtype: int64
```

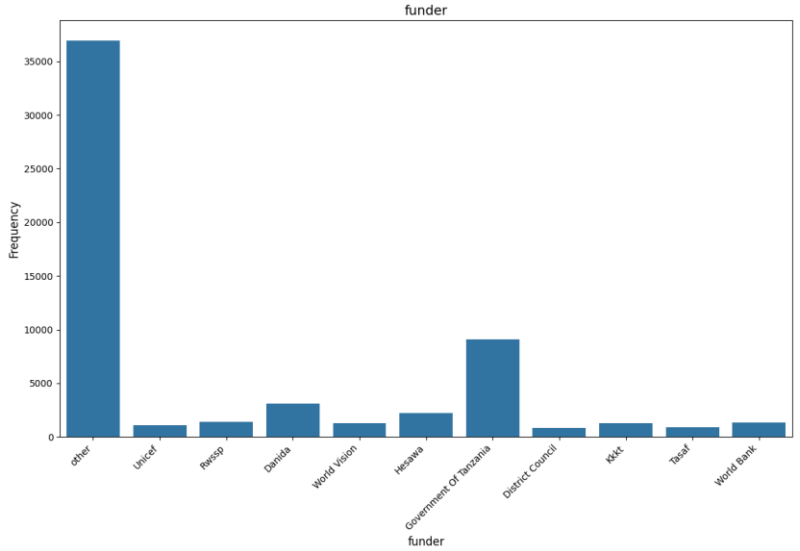
Prepare a dataset

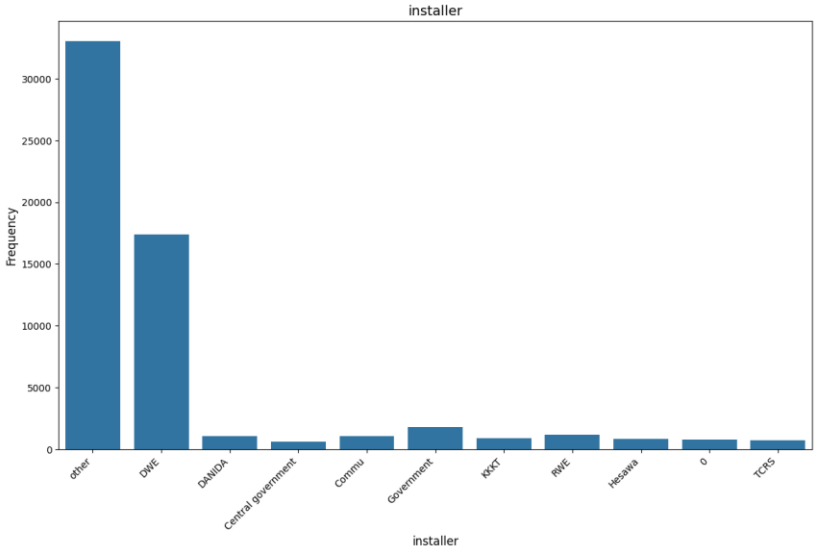
- Reduce the cardinality of “funder” and “installer” features before one-hot encoding

Keep only top 10 unique values. Otherwise, set to “other”

```
# Precompute the top 10
top10_funders = df_filtered[df_filtered['funder'] != 'unknown']['funder'].value_counts().head(10).index
top10_installer = df_filtered[df_filtered['installer'] != 'unknown']['installer'].value_counts().head(10).index

# Reducte cardinality
df_filtered['funder_wrangler'] = df_filtered['funder'].where(df_filtered['funder'].isin(top10_funders), 'other')
df_filtered['installer_wrangler'] = df_filtered['installer'].where(df_filtered['installer'].isin(top10_installer), 'other')
```





Prepare a dataset

- Encode categorical features using OneHotEncoder

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
encoded = encoder.fit_transform(X.select_dtypes(include='object'))

# Create dataframe with encoded columns
encoded_df = pd.DataFrame(encoded, columns=encoder.get_feature_names_out(X.select_dtypes(include='object').columns))

# Combine with numerical columns
X = pd.concat([X.select_dtypes(exclude='object').reset_index(drop=True), encoded_df], axis=1)
```

	basin_Internal	basin_Lake Nyasa	basin_Lake Rukwa	basin_Lake Tanganyika	basin_Lake Victoria	basin_Pangani	basin_Rufiji	basin_Ruvuma / Southern Coast	basin_Wami / Ruvu
0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0
3	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0

Prepare a dataset

- Split the data into training (60%) and testing (40%) (shuffle and stratify by status_group)

```
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.4,random_state=314,shuffle=True,stratify=y)
X_train, X_test = X_train.reset_index(drop=True), X_test.reset_index(drop=True)
print("Functional labels labels in test targets :",y_test[y_test==0].shape[0])
print("Non functional labels labels in test targets :",y_test[y_test==1].shape[0])
print("Functional needs repair labels labels in test targets :",y_test[y_test==2].shape[0])
```

```
Functional labels labels in test targets : 12904
Non functional labels labels in test targets : 1727
Functional needs repair labels labels in test targets : 9129
```

- Standardize numerical features using StandardScaler

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()

numerical_features = X.select_dtypes(include=['int64','float64']).columns

X_train[numerical_features] = sc.fit_transform(X_train[numerical_features])
X_test[numerical_features] = sc.transform(X_test[numerical_features])
```

Model Training

- Use 5-split stratified cross-validation and grid search to find the optimal hyper parameters of each 5 models.
  - Logistic Regression with l2 regularization
  - K-NN
  - Decision Tree
  - Random Forest
  - XGBoost

Results

- The performance of five classification models after fine-tuning
  - Ensemble methods (Random Forest, XGBoost) outperform simpler models
  - Random Forest's strong F1 indicates it better captures minority classes or misclassification-sensitive outcomes.
  - Logistic Regression's low F1 hints that it struggles with nonlinear separability.

Model	Accuracy	F1 Score
Logistic Regression with l2	0.74	0.54
K-NN	0.77	0.66
Decision Tree	0.76	0.64
Random Forest	0.80	0.67
XGBoost	0.79	0.66

Conclusion

- The Random Forest model achieved the highest accuracy in predicting functional and non-functional pumps.
- However, its performance in predicting functional needs repair pumps was relatively poor, likely due to the limited number of records for that class.

Random Forest

	precision	recall	f1-score	support
functional	0.79	0.92	0.85	12904
functional needs repair	0.61	0.28	0.38	1727
non functional	0.86	0.74	0.80	9129
accuracy			0.80	23760
macro avg	0.75	0.65	0.67	23760
weighted avg	0.80	0.80	0.79	23760

Submissions

- To help you track your progress during the competition, each submission is scored against publicly available test data to give a "public score".
- The primary evaluation metric is Classification rate. **Show more**.

Best score	Current rank	Submissions used
0.8014	#5274	1 of 3

Make new submission

You have **2 of 3** submissions left today. Your next submission can be on Oct. 21, 2025 UTC.

THANKS
FOR
LISTENING